UDA 7 · FUNZIONI E STRUTTURE DI DATI SEMPLICI: GLI ARRAY

Simulazione: funzioni e vettori in C e C++

Nome e cognome	Classe	Data

Questa è una prova di allenamento: stessa struttura, stessa durata (**55 minuti**) e stessa difficoltà della verifica vera, ma con domande diverse. Fai tutto senza computer, poi correggiti con le **soluzioni** in fondo. Totale: **100 punti**.

1. Scelta multipla: una sola risposta giusta — 10 punti

a) Quale parola chiave del C restituisce un valore a chi ha chiamato la funzione?

○ `void` ○ `break` ○ `return` ○ `printf`

b) Con `int v[6] = {4, 8, 1, 5, 2, 9};` quale espressione vale 4?

○ `v[1]` ○ `v[0]` ○ `v[6]` ○ `v[4]`

c) Nella chiamata `scambia(&x, &y)` alla funzione si passano:

○ gli indirizzi di x e y
○ delle copie di x e y
○ i nomi delle variabili
○ due vettori

d) In C++, come si scrive un parametro passato per riferimento?

○ `int *a` ○ `int a[]` ○ `const int a` ○ `int &a`

e) La ricerca dicotomica, che dimezza i dati a ogni passo, si può usare:

○ su qualunque vettore
○ solo su vettori con meno di 100 elementi
○ solo su dati ordinati
○ solo su numeri positivi

2. Vero o falso? — 10 punti

V ☐ **F** ☐ Una variabile locale è visibile in tutte le funzioni del programma.

V ☐ **F** ☐ In C una procedura è una funzione di tipo `void` .

V ☐ **F** ☐ Con una sola istruzione `return` una funzione può restituire due valori.

V ☐ **F** ☐ Con `int v[10];` l'istruzione `v[10] = 5;` è un errore, anche se il compilatore può non segnalarlo.

V ☐ **F** ☐ L'ordinamento per selezione fa sempre lo stesso numero di confronti, qualunque sia l'ordine iniziale.

3. Completa la funzione che restituisce il valore più grande del vettore — 8 punti

```
int massimo(int v[], int n) {
    int m = _______________;
    for (int i = 1; i < n; i++)
        if (_______________ > m)
            m = v[i];
    _______________ m;
}
/* nel main, con int voti[7]: */
printf("%d", massimo(voti, ______));
```

4. I puntatori

8 punti

```
int x = 4, y = 9;
int *p = &y;
*p = *p - x;
p = &x;
*p = *p * 2;
```

Alla fine: x = _____ y = _____

In `int *p;` che cosa indica l'asterisco?

In `*p = 0;` che cosa indica l'asterisco?

5. Che cosa stampa? Scrivi l'output esatto sotto ogni frammento

12 punti

In ogni frammento la funzione è definita prima del `main`; le altre righe sono dentro il `main`.

a)

```
int g(int a, int b) { return a - b; }
printf("%d", g(10, g(5, 2)));
```

b)

```
void azzera(int n) { n = 0; }
int k = 8;  azzera(k);
printf("%d", k);
```

c)

```
void azzera(int *n) { *n = 0; }
int k = 8;  azzera(&k);
printf("%d", k);
```

d)

```
int v[6] = {1, 2, 3, 4, 5, 6};  int s = 0;
for (int i = 1; i < 6; i += 2)
    s = s + v[i];
printf("%d", s);
```

6. Traccia la ricerca

8 punti

`cerca(v, n, x)` restituisce l'indice della prima cella uguale a x, oppure -1; `conta(v, n, x)` conta le celle uguali a x. Il vettore è `int v[6] = {5, 1, 6, 2, 6, 3};`

Chiamata	Valore restituito	Numero di confronti fatti
`cerca(v, 6, 6)`		
`cerca(v, 6, 4)`		
`conta(v, 6, 6)`		

7. Trova i bug

8 punti

La funzione dovrebbe contare quante celle del vettore sono uguali a x. Contiene **due errori**: indicali e riscrivi la funzione corretta.

```
int conta(int v[], int n, int x) {
    int c;
    for (int i = 0; i < n; i++)
        if (v[i] = x) c++;
    return c;
}
```

8. Ordinamento: scrivi il vettore dopo ogni passo

14 punti

Il vettore di partenza è `{4, 8, 2, 7, 1}`, da ordinare in modo crescente.

a) Selezione	
Passata 1	
Passata 2	
Passata 3	
Passata 4	

b) Inserzione	
Inserito l'8	
Inserito il 2	
Inserito il 7	
Inserito l'1	

9. Scrivi il programma (in C oppure in C++)

16 punti

Scrivi due funzioni: `int minimo(int v[], int n)`, che restituisce il valore più piccolo del vettore, e `int conta(int v[], int n, int x)`, che conta quante celle sono uguali a x. Nel `main` leggi **8 numeri** in un vettore, poi stampa il minimo e quante volte compare.

Esempio: con 5 3 9 3 7 3 8 4 il programma stampa `Minimo: 3` e `Compare 3 volte`.

10. Domanda aperta

6 punti

Spiega con parole tue come funzionano l'ordinamento per **selezione** e quello per **inserzione**. Quale dei due conviene su un vettore quasi ordinato, e perché?

Soluzioni

Confronta le tue risposte. Voto indicativo = punti ÷ 10. Se hai sbagliato un esercizio, rivedi la lezione indicata tra parentesi.

Es.	Soluzione	Ripassa
1	a) `return`; b) `v[0]`; c) gli indirizzi di x e y; d) `int &a`; e) solo su dati ordinati. (2 punti ciascuna)	Funzioni, parametri, ricerca
2	F (è visibile solo nella sua funzione); V; F (return restituisce un solo valore); V (gli indici vanno da 0 a 9); V (n(n−1)/2 confronti). (2 punti ciascuna)	Parametri, vettori, ordinamento
3	`int m = v[0];` · `if (v[i] > m)` · `return m;` · `massimo(voti, 7)`. (2 punti ciascuno)	Vettori
4	x = **8**, y = **5** (prima y = 9 − 4, poi x = 4 × 2). In `int *p` l'asterisco dice che p è un **puntatore**; in `*p = 0` vuol dire "la cella a cui punta p". (2 punti ciascuna)	Parametri
5	a) **7**: g(5, 2) = 3, g(10, 3) = 7. b) **8**: per valore. c) **0**: per indirizzo. d) **12**: i vale 1, 3, 5 e si sommano 2 + 4 + 6. (3 punti ciascuno)	Funzioni, parametri, vettori
6	`cerca(v, 6, 6)` = **2** con 3 confronti; `cerca(v, 6, 4)` = **−1** con 6 confronti; `conta(v, 6, 6)` = **2**.	Ricerca
7	Errore 1: `c` non è inizializzata, serve `int c = 0;`. Errore 2: `v[i] = x` è un'assegnazione, il confronto è `v[i] == x`. (2 + 2 punti, 4 per la funzione corretta)	Ricerca
8	a) Selezione: `1 8 2 7 4` · `1 2 8 7 4` · `1 2 4 7 8` · `1 2 4 7 8`. b) Inserzione: `4 8 2 7 1` · `2 4 8 7 1` · `2 4 7 8 1` · `1 2 4 7 8`.	Ordinamento
9	Vedi il programma qui sotto. Punti: minimo 4 (deve partire da v[0]), conta 4, lettura con ciclo e & 3, chiamate corrette 3, sintassi e output 2.	Vettori, ricerca
10	Selezione: a ogni passata cerca il minimo della parte non ordinata e lo scambia al suo posto. Inserzione: prende un elemento alla volta, sposta a destra quelli più grandi e lo inserisce tra quelli già ordinati. Su dati quasi ordinati conviene l'**inserzione**: ogni elemento si ferma quasi subito, mentre la selezione fa sempre tutti i confronti.	Ordinamento

```c
#include <stdio.h>
int minimo(int v[], int n) {
    int m = v[0];
    for (int i = 1; i < n; i++)
        if (v[i] < m) m = v[i];
    return m;
}
int conta(int v[], int n, int x) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (v[i] == x) c++;
    return c;
}
```

```c
int main() {
    int a[8];
    for (int i = 0; i < 8; i++)
        scanf("%d", &a[i]);
    int m = minimo(a, 8);
    printf("Minimo: %d\n", m);
    printf("Compare %d volte\n",
           conta(a, 8, m));
    return 0;
}
```